High-Concurrency Booking Engine

MySQL 8.0.16+ schema and query for listing scheduled movie shows at a theatre on a local calendar date. The schema keeps showtimes in the theatre's local time; `theatres.time_zone` records the IANA time zone for display and application-level date handling.

Entities and attributes

Table	Attributes
`customers`	`customer_id` (PK), `full_name`, `email` (unique), `created_at`
`theatres`	`theatre_id` (PK), `name`, `address_line`, `city`, `state_region`, `postal_code`, `country_code`, `time_zone`, `created_at`
`screens`	`screen_id` (PK), `theatre_id` (FK), `name`, `created_at`; screen names are unique within a theatre
`seats`	`seat_id` (PK), `screen_id` (FK), `row_label`, `seat_number`, `seat_type`; a seat position is unique within a screen
`movies`	`movie_id` (PK), `title`, `duration_minutes`, `language`, `certificate`, `created_at`
`shows`	`show_id` (PK), `screen_id` (FK), `movie_id` (FK), `starts_at`, `ends_at`, `status`, `created_at`
`bookings`	`booking_id` (PK), `customer_id` (FK), `show_id` (FK), `status`, `created_at`, `updated_at`
`show_seats`	`show_seat_id` (PK), `show_id` and `seat_id` (FKs, unique together), `inventory_status`, `price`
`booking_seats`	`show_seat_id` (PK and FK to inventory), `booking_id` (FK), `status` (`held` or `booked`), `hold_expires_at`, `price_paid`, `created_at`; the primary key allows at most one claim per show-seat
`payments`	`payment_id` (PK), `booking_id` (FK), `provider`, `provider_payment_id`, `amount`, `currency`, `status`, `created_at`, `updated_at`; provider payment reference is unique
`payment_webhook_events`	`webhook_event_id` (PK), `provider`, `provider_event_id`, `payment_id` (nullable FK), `event_type`, `payload`, `processing_status`, `received_at`, `processed_at`; provider event reference is unique for idempotency

P1 — Schema and sample data

The complete, executable DDL and sample inserts are in `schema.sql`. Run it in MySQL 8.0.16 or newer:

```
mysql -u your_user -p < schema.sql
```

Example rows

The schema script seeds these linked sample records; the IDs are fixed in the sample inserts so the foreign-key relationships can be followed:

Table	Example row
`customers`	`(1, 'Aarav Mehta', 'aarav@example.com')`
`theatres`	`(1, 'Central Cinema', 'Bengaluru', 'Asia/Kolkata')`
`screens`	`(1, 1, 'Screen 1')`
`seats`	`(1, 1, 'A', 1, 'standard')`
`movies`	`(1, 'The Last Orbit', 128, 'English', 'UA')`

Table	Example row
shows	(1, 1, 1, '2026-10-16 18:30:00', 'scheduled')
bookings	(1001, 1, 1, 'held')
show_seats	(1, 1, 1, 'active', 250.00)
booking_seats	(1, 1001, 'held', [12-minute expiry])
payments	(5001, 1001, 'demo_gateway', 'demo_payment_5001', 250.00, 'INR', 'pending')
payment_webhook_events	(1, 'demo_gateway', 'evt_demo_received_5001', 5001, 'payment.pending', 'received')

The full column values and timestamp defaults are specified by the `INSERT` statements in `schema.sql`.

The schema is normalized: seat coordinates are stored once per screen, movie and theatre details are not repeated in show rows, and booking/payment/webhook facts have separate relations. Each column is atomic (1NF); non-key attributes depend on the whole key (2NF); transitive dependencies are kept in their entity tables (3NF). Each non-trivial functional dependency in a base table is determined by a candidate key (BCNF). In particular, seat claims reference a surrogate `show_seat_id` rather than repeating `show_id` alongside `booking_id`. The theatre-to-screen-to-seat and movie-to-show relationships are represented by foreign keys rather than copied descriptive fields.

Concurrency and hold strategy

`show_seats` is the authoritative inventory: create one row for every seat offered for each show. `booking_seats` records the current claim on that inventory row, either a temporary hold or a confirmed booking. A seat is available when it has no claim, or only an expired hold, and its inventory status is `active`. Use Redis for fast, short-lived coordination (for example, a key scoped to `show_id` and `seat_id` with a random hold token and TTL), but never treat Redis as the source of truth. Redis expiry is not a durable booking decision.

For a hold request:

1. Create the booking in a short MySQL transaction, then select the requested `show_seats` rows with `SELECT ... FOR UPDATE`, in ascending `seat_id` order. Verify all requested rows exist and belong to the booking's show. A consistent lock order reduces deadlocks.
2. For every selected seat, reject `unavailable` inventory and any existing booked claim. An existing held claim is reusable only if its `hold_expires_at <= NOW(6)`. If any requested seat is unavailable, roll back the entire transaction.
3. Insert or replace all expired/missing claims in `booking_seats` with status `held` and a common expiry, and mark the booking `held`; commit. Use a conditional update as an additional guard and verify the affected-row count matches the requested seat count.
4. Return the hold token/booking ID and expiry to the client. The token is an authorization secret and should be unguessable; persist only a hash if token-based authorization is used. Payment confirmation and hold release must check the booking ID, current state, and unexpired deadline under row locks.
5. An expiry worker finds expired held claims, locks the associated inventory rows and booking, and releases them only if they are still held and expired. This makes a delayed worker safe: a seat already confirmed or renewed is not released. Run expiry and confirmation transitions transactionally.

On successful payment, process the event and booking in one transaction: insert the unique provider event, lock the booking and its held claims, verify that every hold is still valid, update the claims to `booked`, copy the offered prices into `price_paid`, mark the booking `confirmed`, and mark the payment/event processed. A duplicate `(provider, provider_event_id)` is an already-seen event and must not repeat the transition. If a success event arrives after the hold has expired, do not sell the seat; route the payment to the provider's refund/compensation flow.

The database row locks serialize competing hold attempts for the same inventory rows. The unique key on `show_seats(show_id, seat_id)` ensures one inventory row per show-seat, and the primary key on `booking_seats(show_seat_id)` is the final database guard against two simultaneous claims. Keep transactions short; do not call Redis or a payment provider while holding database locks. Retry deadlocks with a bounded retry policy.

P2 — Shows at a theatre on a date

`queries.sql` contains the executable query. Set the MySQL session variables `@theatre_id` and `@show_date` to the requested theatre ID and local date before running it. The half-open date range preserves use of the `starts_at` index and includes every show starting on that date.

```
SET @theatre_id = 1;
SET @show_date = '2026-10-16';

SELECT
    m.movie_id,
    m.title AS movie_title,
    sh.show_id,
    sh.starts_at AS show_start,
    sh.ends_at AS show_end
FROM theatres AS t
JOIN screens AS sc
    ON sc.theatre_id = t.theatre_id
JOIN shows AS sh
    ON sh.screen_id = sc.screen_id
JOIN movies AS m
    ON m.movie_id = sh.movie_id
WHERE t.theatre_id = @theatre_id
  AND sh.status = 'scheduled'
  AND sh.starts_at >= CAST(@show_date AS DATE)
  AND sh.starts_at < DATE_ADD(CAST(@show_date AS DATE), INTERVAL 1 DAY)
ORDER BY sh.starts_at, m.title, sh.show_id;
```

The showtime is a theatre-local `DATETIME`. If the application accepts dates in another time zone, convert the selected date to the theatre's local calendar date before setting `@show_date`. The `(screen_id, starts_at)` index supports per-screen date-range lookups.

Appendix A - Complete MySQL schema and sample data

```
CREATE DATABASE IF NOT EXISTS booking_engine
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_0900_ai_ci;

USE booking_engine;

CREATE TABLE customers (
    customer_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    full_name VARCHAR(150) NOT NULL,
    email VARCHAR(254) NOT NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    PRIMARY KEY (customer_id),
    UNIQUE KEY uq_customers_email (email)
) ENGINE = InnoDB;

CREATE TABLE theatres (
    theatre_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(150) NOT NULL,
    address_line VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL,
    state_region VARCHAR(100) NOT NULL,
    postal_code VARCHAR(20) NOT NULL,
    country_code CHAR(2) NOT NULL,
    time_zone VARCHAR(64) NOT NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    PRIMARY KEY (theatre_id),
    KEY ix_theatres_city (city, state_region)
) ENGINE = InnoDB;

CREATE TABLE screens (
    screen_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    theatre_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(80) NOT NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    PRIMARY KEY (screen_id),
    UNIQUE KEY uq_screens_theatre_name (theatre_id, name),
    CONSTRAINT fk_screens_theatre
        FOREIGN KEY (theatre_id) REFERENCES theatres (theatre_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;

CREATE TABLE seats (
    seat_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    screen_id BIGINT UNSIGNED NOT NULL,
    row_label VARCHAR(8) NOT NULL,
    seat_number SMALLINT UNSIGNED NOT NULL,
    seat_type ENUM('standard', 'premium', 'accessible', 'recliner') NOT NULL DEFAULT 'standard',
    PRIMARY KEY (seat_id),
    UNIQUE KEY uq_seats_position (screen_id, row_label, seat_number),
    CONSTRAINT fk_seats_screen
        FOREIGN KEY (screen_id) REFERENCES screens (screen_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;

CREATE TABLE movies (
    movie_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    title VARCHAR(200) NOT NULL,
    duration_minutes SMALLINT UNSIGNED NOT NULL,
    language VARCHAR(60) NOT NULL,
    certificate VARCHAR(20) NOT NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    PRIMARY KEY (movie_id),
    CONSTRAINT chk_movies_duration CHECK (duration_minutes > 0)
) ENGINE = InnoDB;

CREATE TABLE shows (
    show_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    screen_id BIGINT UNSIGNED NOT NULL,
```

```
    movie_id BIGINT UNSIGNED NOT NULL,
    starts_at DATETIME NOT NULL COMMENT 'Theatre-local wall-clock time',
    ends_at DATETIME NOT NULL COMMENT 'Theatre-local wall-clock time',
    status ENUM('scheduled', 'cancelled', 'completed') NOT NULL DEFAULT 'scheduled',
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    PRIMARY KEY (show_id),
    UNIQUE KEY uq_shows_screen_start (screen_id, starts_at),
    KEY ix_shows_start_status_screen (starts_at, status, screen_id),
    CONSTRAINT fk_shows_screen
        FOREIGN KEY (screen_id) REFERENCES screens (screen_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_shows_movie
        FOREIGN KEY (movie_id) REFERENCES movies (movie_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT chk_shows_time CHECK (ends_at > starts_at)
) ENGINE = InnoDB;

CREATE TABLE bookings (
    booking_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    customer_id BIGINT UNSIGNED NOT NULL,
    show_id BIGINT UNSIGNED NOT NULL,
    status ENUM('pending', 'held', 'confirmed', 'expired', 'cancelled', 'payment_failed') NOT NULL DEFAULT
'pending',
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    updated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
        ON UPDATE CURRENT_TIMESTAMP(6),
    PRIMARY KEY (booking_id),
    KEY ix_bookings_customer_created (customer_id, created_at),
    KEY ix_bookings_show_status (show_id, status),
    CONSTRAINT fk_bookings_customer
        FOREIGN KEY (customer_id) REFERENCES customers (customer_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_bookings_show
        FOREIGN KEY (show_id) REFERENCES shows (show_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;

CREATE TABLE show_seats (
    show_seat_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    show_id BIGINT UNSIGNED NOT NULL,
    seat_id BIGINT UNSIGNED NOT NULL,
    inventory_status ENUM('active', 'unavailable') NOT NULL DEFAULT 'active',
    price DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (show_seat_id),
    UNIQUE KEY uq_show_seats_show_seat (show_id, seat_id),
    CONSTRAINT fk_show_seats_show
        FOREIGN KEY (show_id) REFERENCES shows (show_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_show_seats_seat
        FOREIGN KEY (seat_id) REFERENCES seats (seat_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT chk_show_seats_price CHECK (price >= 0)
) ENGINE = InnoDB;

CREATE TABLE booking_seats (
    show_seat_id BIGINT UNSIGNED NOT NULL,
    booking_id BIGINT UNSIGNED NOT NULL,
    status ENUM('held', 'booked') NOT NULL DEFAULT 'held',
    hold_expires_at DATETIME(6) NULL,
    price_paid DECIMAL(10, 2) NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    PRIMARY KEY (show_seat_id),
    KEY ix_booking_seats_booking_status (booking_id, status),
    KEY ix_booking_seats_expiry (status, hold_expires_at),
    CONSTRAINT fk_booking_seats_booking
        FOREIGN KEY (booking_id) REFERENCES bookings (booking_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_booking_seats_inventory
        FOREIGN KEY (show_seat_id) REFERENCES show_seats (show_seat_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT chk_booking_seats_state CHECK (
        (status = 'held' AND hold_expires_at IS NOT NULL AND price_paid IS NULL)
        OR
        (status = 'booked' AND hold_expires_at IS NULL AND price_paid IS NOT NULL AND price_paid >= 0)
    )
) ENGINE = InnoDB;
```

```
CREATE TABLE payments (
    payment_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    booking_id BIGINT UNSIGNED NOT NULL,
    provider VARCHAR(40) NOT NULL,
    provider_payment_id VARCHAR(160) NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    currency CHAR(3) NOT NULL,
    status ENUM('pending', 'succeeded', 'failed', 'refunded') NOT NULL DEFAULT 'pending',
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    updated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
        ON UPDATE CURRENT_TIMESTAMP(6),
    PRIMARY KEY (payment_id),
    UNIQUE KEY uq_payments_provider_reference (provider, provider_payment_id),
    KEY ix_payments_booking_created (booking_id, created_at),
    CONSTRAINT fk_payments_booking
        FOREIGN KEY (booking_id) REFERENCES bookings (booking_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT chk_payments_amount CHECK (amount >= 0)
) ENGINE = InnoDB;

CREATE TABLE payment_webhook_events (
    webhook_event_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    provider VARCHAR(40) NOT NULL,
    provider_event_id VARCHAR(160) NOT NULL,
    payment_id BIGINT UNSIGNED NULL,
    event_type VARCHAR(100) NOT NULL,
    payload JSON NOT NULL,
    processing_status ENUM('received', 'processed', 'ignored', 'failed') NOT NULL DEFAULT 'received',
    received_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    processed_at DATETIME(6) NULL,
    PRIMARY KEY (webhook_event_id),
    UNIQUE KEY uq_webhook_provider_event (provider, provider_event_id),
    KEY ix_webhook_payment_received (payment_id, received_at),
    CONSTRAINT fk_webhook_payment
        FOREIGN KEY (payment_id) REFERENCES payments (payment_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;

INSERT INTO customers (customer_id, full_name, email)
VALUES (1, 'Aarav Mehta', 'aarav@example.com');

INSERT INTO theatres (
    theatre_id, name, address_line, city, state_region,
    postal_code, country_code, time_zone
)
VALUES (
    1, 'Central Cinema', '42 Market Road', 'Bengaluru', 'Karnataka',
    '560001', 'IN', 'Asia/Kolkata'
);

INSERT INTO screens (screen_id, theatre_id, name)
VALUES (1, 1, 'Screen 1');

INSERT INTO seats (seat_id, screen_id, row_label, seat_number, seat_type)
VALUES
    (1, 1, 'A', 1, 'standard'),
    (2, 1, 'A', 2, 'standard'),
    (3, 1, 'A', 3, 'premium');

INSERT INTO movies (movie_id, title, duration_minutes, language, certificate)
VALUES
    (1, 'The Last Orbit', 128, 'English', 'UA'),
    (2, 'Monsoon Letters', 114, 'Kannada', 'U');

INSERT INTO shows (show_id, screen_id, movie_id, starts_at, ends_at, status)
VALUES
    (1, 1, 1, '2026-10-16 18:30:00', '2026-10-16 20:38:00', 'scheduled'),
    (2, 1, 2, '2026-10-16 21:15:00', '2026-10-16 23:09:00', 'scheduled');

INSERT INTO bookings (booking_id, customer_id, show_id, status)
VALUES (1001, 1, 1, 'held');

INSERT INTO show_seats (
    show_seat_id, show_id, seat_id, inventory_status, price
)
```

```
VALUES
    (1, 1, 1, 'active', 250.00),
    (2, 1, 2, 'active', 250.00),
    (3, 1, 3, 'active', 350.00),
    (4, 2, 1, 'active', 220.00),
    (5, 2, 2, 'active', 220.00),
    (6, 2, 3, 'active', 320.00);

INSERT INTO booking_seats (
    show_seat_id, booking_id, status, hold_expires_at, price_paid
)
VALUES (1, 1001, 'held', CURRENT_TIMESTAMP(6) + INTERVAL 12 MINUTE, NULL);

INSERT INTO payments (
    payment_id, booking_id, provider, provider_payment_id,
    amount, currency, status
)
VALUES (
    5001, 1001, 'demo_gateway', 'demo_payment_5001',
    250.00, 'INR', 'pending'
);

INSERT INTO payment_webhook_events (
    webhook_event_id, provider, provider_event_id, payment_id,
    event_type, payload, processing_status
)
VALUES (
    1, 'demo_gateway', 'evt_demo_received_5001', 5001,
    'payment.pending',
    JSON_OBJECT('id', 'evt_demo_received_5001', 'type', 'payment.pending'),
    'received'
);
```

Appendix B - Theatre/date showtime query

```
SET @theatre_id = 1;
SET @show_date = '2026-10-16';

SELECT
    m.movie_id,
    m.title AS movie_title,
    sh.show_id,
    sh.starts_at AS show_start,
    sh.ends_at AS show_end
FROM theatres AS t
JOIN screens AS sc
    ON sc.theatre_id = t.theatre_id
JOIN shows AS sh
    ON sh.screen_id = sc.screen_id
JOIN movies AS m
    ON m.movie_id = sh.movie_id
WHERE t.theatre_id = @theatre_id
  AND sh.status = 'scheduled'
  AND sh.starts_at >= CAST(@show_date AS DATE)
  AND sh.starts_at < DATE_ADD(CAST(@show_date AS DATE), INTERVAL 1 DAY)
ORDER BY sh.starts_at, m.title, sh.show_id;
```